Universidad Nacional Mayor de San Marcos

Universidad del Perú. Decana de América

Facultad de Ciencias Matemáticas



Sistema Automático de Control de Asistencia para Docentes

Curso: Base de datos

Docente: Ing. Wiler Arturo Ponce Benites

Integrantes:
Roque Huamani, Alejandro Harold
Palomino Gutierres, Joel Raul
Sanchez Ojeda, Nicolas
Bartolo Huere, Edson

17 de junio de 2026

PARTE 1: DESARROLLO DE LA BASE DE DATOS

1. Reglas de Negocio

Las reglas de negocio definen las restricciones y comportamientos lógicos que gobiernan el sistema:

Código	Regla de Negocio	Descripción
RN01	Identificación única del docente	Cada docente es identificado por un código alfanumérico único de exactamente 6 caracteres (solo letras mayúsculas y números).
RN02	DNI válido	El DNI del docente debe ser numérico y tener exactamente 8 dígitos.
RN03	Nombres de cursos estandarizados	Los nombres de los cursos se almacenan siempre en MAYÚSCULAS para garantizar uniformidad.
RN04	Código de curso estandarizado	El código del curso, si se asigna, debe ser alfanumérico (solo mayúsculas y números) con un máximo de 6 caracteres.
RN05	Créditos válidos	Los créditos de un curso deben estar en el rango permitido: 2, 3, 4, 5 o 6.
RN06	Periodo académico activo	Solo puede existir un periodo académico marcado como activo (`EsActivo = 1`) a la vez. Todos los registros de asistencia y horarios se asocian al periodo vigente.
RN07	Sin superposición de horarios	El sistema no permite asignar a un mismo docente dos clases que se superpongan en día y hora dentro del mismo periodo académico.
RN08	Horario único por bloque	La combinación de DocenteId + CursoId + DiaSemana + HoraInicio + PeriodoId debe ser única (restricción UNIQUE compuesta en la tabla HorarioClases).
RN09	Registro secuencial de asistencia	Un docente debe registrar primero su ENTRADA. Solo puede registrar SALIDA si existe una entrada previa sin cerrar para el mismo curso en el mismo día.
RN10	Cierre obligatorio antes de nueva sesión	Si un docente tiene dos cursos el mismo día (ej. mañana y tarde), debe cerrar la sesión del primer curso (registrar salida) antes de que el sistema le permita registrar asistencia al segundo curso.
RN11	Timeout de seguridad	Las interfaces de registro tienen un temporizador de 20 segundos. Si no se confirma la acción, se cancela automáticamente y el sistema regresa al estado inicial.

2. Identificación del Diccionario de Datos

Tabla: `Docentes`

Almacena la información personal y administrativa de cada docente de la EPCC.

#	Columna	Tipo de Dato	Nulable	Restricción	Descripción
1	`Id`	INT	NO	PK, IDENTITY(1,1)	Identificador único autoincremental del docente.
2	`Codigo`	VARCHAR(20)	NO	UNIQUE, CHECK (LEN=6, solo A-Z y 0-9)	Código único de identificación del docente (Ej: DOC001).
3	`Nombre`	NVARCHAR(100)	NO	—	Nombres del docente.
4	`Apellido`	NVARCHAR(100)	NO	—	Apellidos del docente.
5	`DNI`	NVARCHAR(20)	SÍ	CHECK (LEN=8, solo dígitos)	Documento Nacional de Identidad.
6	`Clase`	VARCHAR(50)	SÍ	—	Tipo de dedicación docente (Ej: “TP 20 hrs.”, “TP 40 hrs.”).
7	`Categoria`	VARCHAR(50)	SÍ	—	Categoría académica del docente (Ej: “Principal”, “Asociado”).
8	`Departamento`	VARCHAR(100)	SÍ	—	Departamento académico al que pertenece (Ej: “Computación Científica”, “Matemática”).
9	`Correo`	VARCHAR(100)	SÍ	—	Correo electrónico institucional del docente.

Tabla: `Cursos`

Catálogo de asignaturas o cursos ofertados por la escuela.

#	Columna	Tipo de Dato	Nulable	Restricción	Descripción
1	`Id`	INT	NO	PK, IDENTITY(1,1)	Identificador único autoincremental del curso.
2	`Nombre`	VARCHAR(150)	NO	UNIQUE, CHECK (debe ser MAYÚSCULAS)	Nombre oficial del curso (Ej: "BASE DE DATOS").
3	`Codigo`	VARCHAR(20)	SÍ	CHECK (máx. 6 chars, solo A-Z y 0-9)	Código del curso según malla (Ej: "INF201").
4	`Creditos`	INT	SÍ	CHECK (valor ∈ {2, 3, 4, 5, 6})	Cantidad de créditos del curso.
5	`Tipo`	VARCHAR(5)	SÍ	—	Tipo del curso: "O" = Obligatorio, "E" = Electivo.
6	`Escuela`	VARCHAR(100)	SÍ	—	Escuela profesional a la que pertenece el curso.
7	`MallaCurricular`	VARCHAR(20)	SÍ	—	Identificador de la malla curricular vigente del curso.

Tabla: `Periodos`

Catálogo de periodos o semestres académicos.

#	Columna	Tipo de Dato	Nulable	Restricción	Descripción
1	`Id`	INT	NO	PK, IDENTITY(1,1)	Identificador único autoincremental del periodo.
2	`Nombre`	VARCHAR(50)	NO	UNIQUE	Nombre descriptivo del periodo (Ej: "2026-I", "2026-II").
3	`EsActivo`	BIT	NO	DEFAULT 0	Indica si el periodo es el vigente. Solo uno debe ser 1.

Tabla: `HorarioClases`

Define la programación semanal de clases de cada docente: qué curso dicta, qué día y a qué hora.

#	Columna	Tipo de Dato	Nulable	Restricción	Descripción
1	`Id`	INT	NO	PK, IDENTITY(1,1)	Identificador único autoincremental del bloque horario.
2	`DocenteId`	INT	NO	FK → Docentes(Id)	Referencia al docente asignado.
3	`CursoId`	INT	NO	FK → Cursos(Id)	Referencia al curso que se dicta.
4	`DiaSemana`	INT	NO	—	Día de la semana (1=Lunes, 2=Martes, 3=Miércoles, 4=Jueves, 5=Viernes, 6=Sábado, 7=Domingo).
5	`HoraInicio`	TIME(7)	NO	—	Hora exacta de inicio de la clase.
6	`HoraFin`	TIME(7)	NO	—	Hora exacta de fin de la clase.
7	`PeriodoId`	INT	SÍ	FK → Periodos(Id)	Periodo académico al que pertenece este horario.

Restricción UNIQUE compuesta: `(DocenteId, CursoId, DiaSemana, HoraInicio, PeriodoId)` — Impide registros duplicados del mismo bloque horario.

Tabla: `RegistroAsistencia`

Tabla transaccional que almacena cada evento de entrada y salida de los docentes.

#	Columna	Tipo de Dato	Nulable	Restricción	Descripción
1	`Id`	INT	NO	PK, IDENTITY(1,1)	Identificador único autoincremental del registro.
2	`DocenteId`	INT	SÍ	FK → Docentes(Id)	Referencia al docente que registra asistencia.
3	`CursoId`	INT	SÍ	FK → Cursos(Id)	Referencia al curso al que asiste.
4	`FechaHora`	DATETIME	SÍ	—	Fecha y hora exacta del registro de ENTRADA.
5	`HoraSalida`	DATETIME	SÍ	—	Fecha y hora de la SALIDA. Si es NULL, la sesión está abierta.
6	`PeriodoId`	INT	SÍ	FK → Periodos(Id)	Periodo académico en el cual se registra la asistencia.

3. Modelo Lógico (Modelo Entidad-Relación)

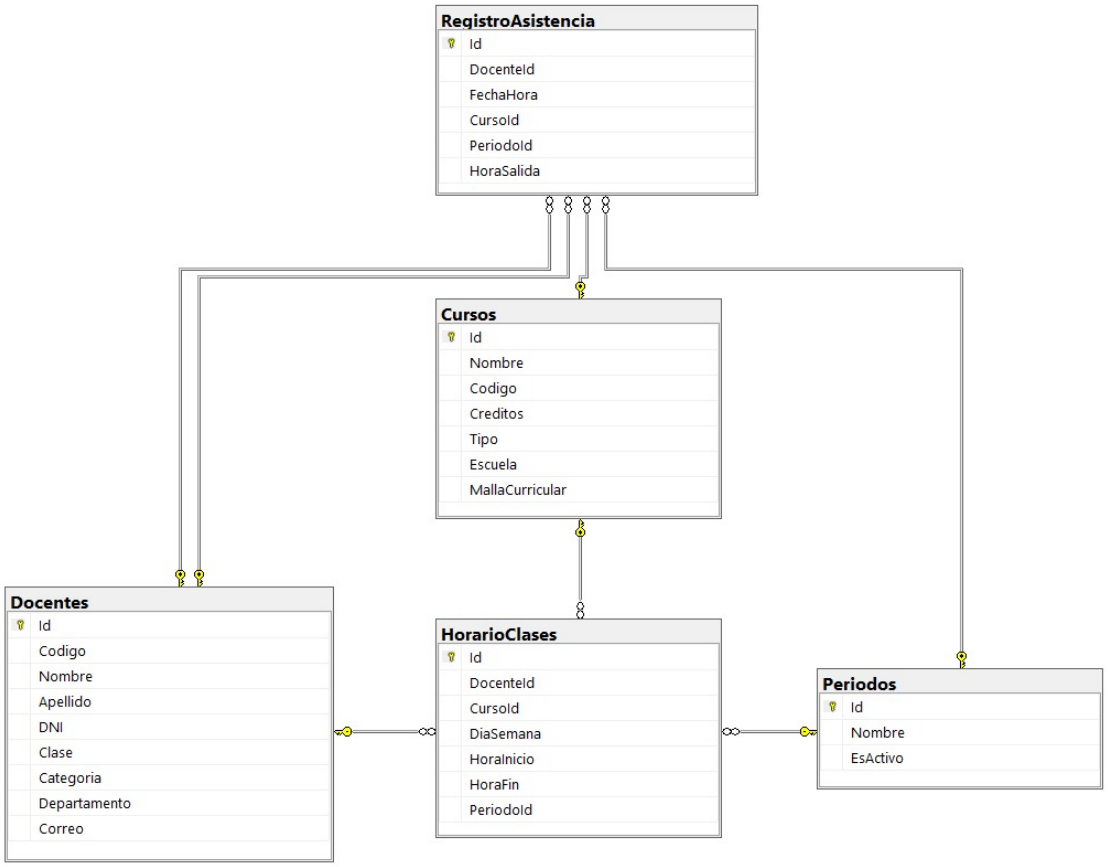


Figura 1: diagrama

Relaciones:

- **Docentes → HorarioClases:** Un docente tiene muchos bloques horarios (1:N).
- **Docentes → RegistroAsistencia:** Un docente genera muchos registros de asistencia (1:N).
- **Cursos → HorarioClases:** Un curso puede estar en muchos bloques horarios (1:N).
- **Cursos → RegistroAsistencia:** Un curso aparece en muchos registros de asistencia (1:N).
- **Periodos → HorarioClases:** Un periodo contiene muchos horarios (1:N).
- **Periodos → RegistroAsistencia:** Un periodo contiene muchos registros (1:N).

4. Scripts

4.1 Script DDL — Creación de la Base de Datos y Tablas

```
-- ============================================================
-- SCRIPT DDL: Creaci n de la Base de Datos AsistenciaFacultad
-- Motor: SQL Server / SQL Server Express
-- ============================================================

CREATE DATABASE AsistenciaFacultad;
GO
USE AsistenciaFacultad;
GO

-- Tabla: Docentes
CREATE TABLE [dbo].[Docentes](
    [Id]            INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    [Codigo]        VARCHAR(20)   NOT NULL,
    [Nombre]        NVARCHAR(100) NOT NULL,
    [Apellido]      NVARCHAR(100) NOT NULL,
    [DNI]           NVARCHAR(20)  NULL,
    [Clase]         VARCHAR(50)   NULL,
    [Categoria]     VARCHAR(50)   NULL,
    [Departamento]  VARCHAR(100)  NULL,
    [Correo]        VARCHAR(100)  NULL
);

-- Tabla: Cursos
CREATE TABLE [dbo].[Cursos](
    [Id]              INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    [Nombre]          VARCHAR(150)  NOT NULL,
    [Codigo]          VARCHAR(20)   NULL,
    [Creditos]        INT           NULL,
    [Tipo]            VARCHAR(5)    NULL,
    [Escuela]         VARCHAR(100)  NULL,
    [MallaCurricular] VARCHAR(20)   NULL
);

-- Tabla: Periodos
CREATE TABLE [dbo].[Periodos](
    [Id]       INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    [Nombre]   VARCHAR(50) NOT NULL UNIQUE,
    [EsActivo] BIT DEFAULT 0
);

-- Tabla: HorarioClases
CREATE TABLE [dbo].[HorarioClases](
    [Id]          INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    [DocenteId]   INT      NOT NULL,
    [CursoId]     INT      NOT NULL,
    [DiaSemana]   INT      NOT NULL,
    [HoraInicio]  TIME(7)  NOT NULL,
    [HoraFin]     TIME(7)  NOT NULL,
    [PeriodoId]   INT      NULL
);

-- Tabla: RegistroAsistencia
CREATE TABLE [dbo].[RegistroAsistencia](
    [Id]          INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
```

```
    [DocenteId]   INT      NULL,
    [FechaHora]   DATETIME NULL,
    [HoraSalida] DATETIME NULL,
    [CursoId]     INT      NULL,
    [PeriodoId]   INT      NULL
);
```

4.2 Script de Restricciones (Constraints)

```
-- ======== UNIQUE ========
ALTER TABLE [dbo].[Docentes] ADD CONSTRAINT UQ_Docentes_Codigo UNIQUE ([
   Codigo]);
ALTER TABLE [dbo].[Cursos]   ADD CONSTRAINT UQ_Cursos_Nombre   UNIQUE ([
   Nombre]);

-- ======== CHECK ========
ALTER TABLE [dbo].[Docentes] ADD CONSTRAINT CK_Docentes_Codigo
    CHECK (LEN(Codigo) = 6 AND Codigo NOT LIKE '%[^A-Z0-9]%' COLLATE
       Latin1_General_BIN);

ALTER TABLE [dbo].[Docentes] ADD CONSTRAINT CK_Docentes_DNI
    CHECK (LEN(DNI) = 8 AND DNI NOT LIKE '%[^0-9]%');

ALTER TABLE [dbo].[Cursos] ADD CONSTRAINT CK_Cursos_Codigo
    CHECK (Codigo IS NULL OR (LEN(Codigo) <= 6 AND Codigo NOT LIKE '%[^A
       -Z0-9]%' COLLATE Latin1_General_BIN));

ALTER TABLE [dbo].[Cursos] ADD CONSTRAINT CK_Cursos_Nombre
    CHECK (Nombre = UPPER(Nombre) COLLATE SQL_Latin1_General_CP1_CS_AS);

ALTER TABLE [dbo].[Cursos] ADD CONSTRAINT CK_Cursos_Creditos
    CHECK (Creditos IS NULL OR Creditos IN (2, 3, 4, 5, 6));

-- ======== FOREIGN KEYS ========
ALTER TABLE [dbo].[RegistroAsistencia] ADD CONSTRAINT
   FK_Registro_Docente
    FOREIGN KEY ([DocenteId]) REFERENCES [dbo].[Docentes]([Id]);

ALTER TABLE [dbo].[RegistroAsistencia] ADD CONSTRAINT FK_Registro_Curso
    FOREIGN KEY ([CursoId]) REFERENCES [dbo].[Cursos]([Id]);

ALTER TABLE [dbo].[RegistroAsistencia] ADD CONSTRAINT
   FK_Registro_Periodo
    FOREIGN KEY ([PeriodoId]) REFERENCES [dbo].[Periodos]([Id]);

ALTER TABLE [dbo].[HorarioClases] ADD CONSTRAINT FK_Horario_Docente
    FOREIGN KEY ([DocenteId]) REFERENCES [dbo].[Docentes]([Id]);

ALTER TABLE [dbo].[HorarioClases] ADD CONSTRAINT FK_Horario_Curso
    FOREIGN KEY ([CursoId]) REFERENCES [dbo].[Cursos]([Id]);

ALTER TABLE [dbo].[HorarioClases] ADD CONSTRAINT FK_Horario_Periodo
    FOREIGN KEY ([PeriodoId]) REFERENCES [dbo].[Periodos]([Id]);

ALTER TABLE [dbo].[HorarioClases] ADD CONSTRAINT UQ_Horario_Unico
    UNIQUE ([DocenteId], [CursoId], [DiaSemana], [HoraInicio], [
       PeriodoId]);
```

4.3 Script DML — Datos de Prueba (Seed Data)

```
-- ======== DOCENTES ========
SET IDENTITY_INSERT [dbo].[Docentes] ON;
INSERT INTO [dbo].[Docentes] ([Id],[Codigo],[Nombre],[Apellido],[DNI],[
    Clase],[Categoria],[Departamento],[Correo]) VALUES
(1,  'DOC001', 'ALAN',    'TURING',   '12345678', 'TP 20 hrs.', '
    Principal', 'Computaci n  Cient fica',    'alan.turing@unmsm.edu.pe'
    ),
(2,  'DOC002', 'ADA',     'LOVELACE', '87654321', 'TP 20 hrs.', '
    Principal', 'Computaci n  Cient fica',    'ada.lovelace@unmsm.edu.pe
    '),
(3,  'DOC003', 'MARIE',   'CURIE',    '11223344', 'TP 20 hrs.', '
    Asociado',  'Matem tica',                 'marie.curie@unmsm.edu.pe')
    ,
(4,  'DOC004', 'ALBERT',  'EINSTEIN', '44332211', 'TP 40 hrs.', '
    Principal', 'Matem tica',                 'albert.einstein@unmsm.edu.
    pe'),
(5,  'DOC005', 'ISAAC',   'NEWTON',   '55667788', 'TP 40 hrs.', '
    Asociado',  'Matem tica',                 'isaac.newton@unmsm.edu.pe'
    ),
(6,  'DOC006', 'NIKOLA',  'TESLA',    '88776655', 'TP 20 hrs.', '
    Principal', 'Investigaci n  Operativa',   'nikola.tesla@unmsm.edu.pe'
    ),
(7,  'DOC007', 'GRACE',   'HOPPER',   '99887766', 'TP 40 hrs.', '
    Asociado',  'Computaci n  Cient fica',    'grace.hopper@unmsm.edu.pe
    '),
(8,  'DOC008', 'STEPHEN', 'HAWKING',  '66778899', 'TP 40 hrs.', '
    Principal', 'Estad stica',                'stephen.hawking@unmsm.edu.
    pe'),
(9,  'DOC009', 'RICHARD', 'FEYNMAN',  '22334455', 'TP 20 hrs.', '
    Asociado',  'Estad stica',                'richard.feynman@unmsm.edu.
    pe'),
(10, 'DOC010', 'CARL',    'SAGAN',    '55443322', 'TP 20 hrs.', '
    Principal', 'Investigaci n  Operativa',   'carl.sagan@unmsm.edu.pe');
SET IDENTITY_INSERT [dbo].[Docentes] OFF;

-- ======== CURSOS ========
SET IDENTITY_INSERT [dbo].[Cursos] ON;
INSERT INTO [dbo].[Cursos] ([Id],[Nombre],[Codigo],[Creditos],[Tipo])
    VALUES
(1,  'C LCULO I',                'MAT101', 4, 'O'),
(2,  'C LCULO II',               'MAT102', 4, 'O'),
(3,  'F SICA I',                 'FIS101', 4, 'O'),
(4,  'PROGRAMACI N B SICA',      'INF101', 5, 'O'),
(5,  'BASE DE DATOS',            'INF201', 5, 'O'),
(6,  'INTELIGENCIA ARTIFICIAL',  'INF301', 6, 'E'),
(7,  'ESTRUCTURAS DE DATOS',     'INF102', 5, 'O'),
(8,  'SISTEMAS OPERATIVOS',      'INF202', 4, 'O'),
(9,  'REDES DE COMPUTADORAS',    'INF302', 4, 'E'),
(10, 'COMPUTACI N  CU NTICA',    'INF401', 6, 'E');
SET IDENTITY_INSERT [dbo].[Cursos] OFF;
```

5. Procedimientos Almacenados

SP 1: Registrar Asistencia (Entrada o Salida automática)

```
CREATE PROCEDURE sp_RegistrarAsistencia
    @DocenteId INT,
    @CursoId   INT,
    @PeriodoId INT = NULL
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @RegistroExistente INT = NULL;

    -- Verificar si hay una entrada abierta (sin salida) para hoy y este
        curso
    SELECT TOP 1 @RegistroExistente = Id
    FROM RegistroAsistencia
    WHERE DocenteId = @DocenteId
      AND CursoId   = @CursoId
      AND CAST(FechaHora AS DATE) = CAST(GETDATE() AS DATE)
      AND HoraSalida IS NULL
    ORDER BY FechaHora DESC;

    IF @RegistroExistente IS NOT NULL
    BEGIN
        -- Ya existe entrada abierta -> registrar SALIDA
        UPDATE RegistroAsistencia
        SET HoraSalida = GETDATE()
        WHERE Id = @RegistroExistente;

        PRINT 'SALIDA registrada correctamente.';
    END
    ELSE
    BEGIN
        -- No hay entrada abierta -> registrar ENTRADA
        INSERT INTO RegistroAsistencia (DocenteId, CursoId, FechaHora,
            PeriodoId)
        VALUES (@DocenteId, @CursoId, GETDATE(), @PeriodoId);

        PRINT 'ENTRADA registrada correctamente.';
    END
END
GO
```

SP 2: Obtener horarios del día para un docente

```
CREATE PROCEDURE sp_ObtenerHorariosDelDia
    @DocenteId  INT,
    @DiaSemana  INT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT h.Id, h.CursoId, c.Nombre AS CursoNombre,
           h.HoraInicio, h.HoraFin, h.DiaSemana
    FROM HorarioClases h
    INNER JOIN Cursos c ON h.CursoId = c.Id
    INNER JOIN Periodos p ON h.PeriodoId = p.Id
    WHERE h.DocenteId = @DocenteId
      AND h.DiaSemana = @DiaSemana
      AND p.EsActivo  = 1
    ORDER BY h.HoraInicio ASC;
END
GO
```

SP 3: Verificar conflicto de horario antes de insertar

```
CREATE PROCEDURE sp_VerificarConflictoHorario
    @DocenteId  INT,
    @DiaSemana  INT,
    @HoraInicio TIME,
    @HoraFin    TIME,
    @PeriodoId  INT,
    @ExcluirId  INT = 0,
    @HayConflicto BIT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @Conflictos INT;

    SELECT @Conflictos = COUNT(*)
    FROM HorarioClases
    WHERE DocenteId = @DocenteId
      AND DiaSemana = @DiaSemana
      AND PeriodoId = @PeriodoId
      AND Id <> @ExcluirId
      AND ((@HoraInicio >= HoraInicio AND @HoraInicio < HoraFin)
        OR (@HoraFin > HoraInicio AND @HoraFin <= HoraFin)
        OR (@HoraInicio <= HoraInicio AND @HoraFin >= HoraFin));

    SET @HayConflicto = CASE WHEN @Conflictos > 0 THEN 1 ELSE 0 END;
END
GO
```

6. Funciones

Función 1: Calcular horas trabajadas de un registro

```
CREATE FUNCTION fn_CalcularHorasTrabajadas (
    @RegistroId INT
)
RETURNS DECIMAL(5,2)
AS
BEGIN
    DECLARE @Horas DECIMAL(5,2) = 0;

    SELECT @Horas = CASE
        WHEN HoraSalida IS NOT NULL
        THEN CAST(DATEDIFF(MINUTE, FechaHora, HoraSalida) AS DECIMAL
           (5,2)) / 60.0
        ELSE 0
    END
    FROM RegistroAsistencia
    WHERE Id = @RegistroId;

    RETURN @Horas;
END
GO
```

Función 2: Obtener total de asistencias de un docente en un periodo

```
CREATE FUNCTION fn_TotalAsistenciasPeriodo (
    @DocenteId INT,
    @PeriodoId INT
)
RETURNS INT
AS
BEGIN
    DECLARE @Total INT = 0;

    SELECT @Total = COUNT(*)
    FROM RegistroAsistencia
    WHERE DocenteId = @DocenteId
      AND PeriodoId = @PeriodoId
      AND HoraSalida IS NOT NULL;

    RETURN @Total;
END
GO
```

Función 3: Obtener nombre del día de la semana en español

```
CREATE FUNCTION fn_NombreDiaSemana (
    @DiaSemana INT
)
RETURNS VARCHAR(15)
AS
BEGIN
    RETURN CASE @DiaSemana
        WHEN 1 THEN 'Lunes'
        WHEN 2 THEN 'Martes'
        WHEN 3 THEN 'Mi rcoles'
        WHEN 4 THEN 'Jueves'
        WHEN 5 THEN 'Viernes'
        WHEN 6 THEN 'S bado'
        WHEN 7 THEN 'Domingo'
        ELSE 'Desconocido'
    END;
END
GO
```

7. Cursores

Cursor 1: Auditoría de entradas sin salida (registros huérfanos)

```
-- Cursor que recorre todos los registros de asistencia donde el docente
-- olvid  marcar su salida en d as anteriores al actual.

DECLARE @RegId INT, @DocenteNombre NVARCHAR(200), @CursoNombre VARCHAR
   (150), @Fecha DATETIME;

DECLARE cursor_FaltasSalida CURSOR FOR
    SELECT r.Id,
           d.Nombre + ' ' + d.Apellido AS NombreCompleto,
           c.Nombre AS Curso,
           r.FechaHora
    FROM RegistroAsistencia r
    INNER JOIN Docentes d ON r.DocenteId = d.Id
    INNER JOIN Cursos c   ON r.CursoId   = c.Id
    WHERE r.HoraSalida IS NULL
      AND CAST(r.FechaHora AS DATE) < CAST(GETDATE() AS DATE)
    ORDER BY r.FechaHora ASC;

OPEN cursor_FaltasSalida;
FETCH NEXT FROM cursor_FaltasSalida INTO @RegId, @DocenteNombre,
   @CursoNombre, @Fecha;

WHILE @@FETCH_STATUS = 0
BEGIN
    PRINT 'ALERTA: El docente "' + @DocenteNombre
        + '" NO registr  su salida del curso "' + @CursoNombre
        + '" el d a ' + CONVERT(VARCHAR, @Fecha, 103)
        + ' (Registro #' + CAST(@RegId AS VARCHAR) + ')';

    FETCH NEXT FROM cursor_FaltasSalida INTO @RegId, @DocenteNombre,
       @CursoNombre, @Fecha;
END

CLOSE cursor_FaltasSalida;
DEALLOCATE cursor_FaltasSalida;
```

Cursor 2: Reporte de horas acumuladas por docente en el periodo activo

```
-- Cursor que calcula e imprime las horas totales acumuladas por cada
   docente
-- en el periodo acad mico activo.

DECLARE @DocId INT, @DocNombre NVARCHAR(200), @TotalMinutos INT;

DECLARE cursor_HorasDocente CURSOR FOR
    SELECT d.Id, d.Nombre + ' ' + d.Apellido
    FROM Docentes d
    ORDER BY d.Apellido, d.Nombre;

OPEN cursor_HorasDocente;
FETCH NEXT FROM cursor_HorasDocente INTO @DocId, @DocNombre;

WHILE @@FETCH_STATUS = 0
BEGIN
    SELECT @TotalMinutos = ISNULL(SUM(DATEDIFF(MINUTE, FechaHora,
       HoraSalida)), 0)
    FROM RegistroAsistencia r
    INNER JOIN Periodos p ON r.PeriodoId = p.Id
    WHERE r.DocenteId = @DocId
      AND r.HoraSalida IS NOT NULL
      AND p.EsActivo = 1;

    PRINT @DocNombre + ': '
        + CAST(@TotalMinutos / 60 AS VARCHAR) + ' horas '
        + CAST(@TotalMinutos % 60 AS VARCHAR) + ' minutos';

    FETCH NEXT FROM cursor_HorasDocente INTO @DocId, @DocNombre;
END

CLOSE cursor_HorasDocente;
DEALLOCATE cursor_HorasDocente;
```

PARTE 2: DESARROLLO DEL SOFTWARE

1. Ingeniería de Requisitos

1.1 Requisitos Funcionales (RF)

ID	Requisito Funcional	Descripción
RF01	Identificación del docente	El sistema debe permitir al docente identificarse ingresando su código único alfanumérico. El sistema busca en la tabla Docentes y muestra el nombre completo al encontrarlo.
RF02	Detección automática de horario	Al ingresar el código, el sistema consulta la tabla HorarioClases filtrando por el día actual y el periodo activo, y muestra automáticamente los cursos que el docente tiene programados para ese día.
RF03	Registro de entrada	El sistema registra la hora de entrada (INSERT en RegistroAsistencia con FechaHora = fecha/hora actual) cuando el docente confirma su asistencia a un curso.
RF04	Registro de salida	Si el docente ya tiene una entrada registrada sin salida para un curso del día, el sistema detecta esto y registra la salida (UPDATE HoraSalida).
RF05	Validación secuencial	Si el docente tiene múltiples cursos en el día, debe cerrar (registrar salida) el primer curso antes de poder registrar entrada al siguiente.
RF06	Gestión CRUD de docentes	El módulo administrativo permite crear, leer, actualizar y eliminar registros de docentes con todos sus campos (Código, Nombre, Apellido, DNI, Clase, Categoría, Departamento, Correo).
RF07	Gestión CRUD de cursos	El módulo administrativo permite crear, leer, actualizar y eliminar registros de cursos con todos sus campos (Nombre, Código, Créditos, Tipo, Escuela, Malla Curricular).
RF08	Gestión CRUD de horarios	El módulo administrativo permite crear, leer, actualizar y eliminar bloques horarios, asignando docente, curso, día, hora inicio y hora fin.
RF09	Gestión de periodos académicos	El administrador puede crear periodos académicos y activar/desactivar el periodo vigente.
RF10	Importación masiva por CSV	El sistema permite importar docentes y cursos de forma masiva mediante archivos CSV.
RF11	Exportación de reportes	El sistema genera reportes exportables a PDF y CSV con los registros de asistencia.
RF12	Autenticación de administrador	El acceso al módulo administrativo requiere usuario y contraseña.

1.2 Requisitos No Funcionales (RNF)

ID	Requisito No Funcional	Descripción
RNF01	Plataforma	Aplicación de escritorio desarrollada en C# con Windows Forms (.NET Framework 4.7.2).
RNF02	Motor de BD	SQL Server Express con conexión mediante ADO.NET (System.Data.SqlClient).
RNF03	Usabilidad tipo kiosco	La interfaz del docente opera sin bordes de ventana (estilo kiosco), con fuentes grandes y operable completamente con teclado numérico o lector de código de barras.
RNF04	Timeout de seguridad	La ventana de registro se cierra automáticamente tras 20 segundos de inactividad para evitar sesiones abiertas.
RNF05	Arquitectura	El sistema implementa un patrón lógico de mínimo 4 capas (Entidades, Acceso a Datos, Lógica de Negocio, Presentación).
RNF06	Rendimiento	El registro de asistencia completo (desde ingreso de código hasta confirmación) debe completarse en menos de 5 segundos.
RNF07	Interfaz visual	Uso de la librería Guna.UI2 para componentes visuales modernos con bordes redondeados y estilos profesionales.

1.3 Lista de Procesos

#	Proceso	Actor	Descripción
P01	Registro de asistencia docente	Docente	Flujo principal del sistema. El docente ingresa su código, el sistema detecta sus cursos del día, y registra entrada o salida.
P02	Gestión de docentes	Administrador	CRUD completo de la tabla Docentes, incluyendo importación CSV.
P03	Gestión de cursos	Administrador	CRUD completo de la tabla Cursos, incluyendo importación CSV.
P04	Gestión de horarios	Administrador	CRUD de la tabla HorarioClases con validación de conflictos de superposición.
P05	Gestión de periodos	Administrador	Crear periodos y activar/desactivar el periodo vigente.
P06	Generación de reportes	Administrador	Consultar registros de asistencia con filtros y exportar a PDF/CSV.
P07	Inicio de sesión	Administrador	Autenticación con credenciales para acceder al panel administrativo.

1.4 Especificación de Procesos

Proceso P01 — Registro de Asistencia Docente (Flujo detallado):

1. El docente se acerca a la terminal y digita su código (Ej: DOC001) en el campo de texto.
2. El sistema busca el código en la tabla `Docentes`.
   - Si **no existe**: muestra mensaje de error y vuelve al paso 1.
   - Si **existe**: obtiene `Id`, `Nombre` y `Apellido` del docente.
3. El sistema calcula el día de la semana actual (Lunes=1, ..., Domingo=7).
4. Consulta la tabla `HorarioClases` filtrando por `DocenteId`, `DiaSemana` y `PeriodoId` activo.
5. Consulta la tabla `RegistroAsistencia` para detectar entradas abiertas (sin `HoraSalida`) del día actual.
6. **Decisión:**
   - Si hay entradas abiertas (salidas pendientes): muestra la lista con ítems marcados "[SALIDA PENDIENTE]" y prioriza el registro de salida.
   - Si no hay entradas abiertas y hay cursos sin registrar: muestra los cursos disponibles con "[INGRESO]" para confirmar la entrada.
   - Si no hay horarios para hoy: abre el formulario en modo manual (fallback) donde el docente busca y selecciona un curso.
7. El docente selecciona el curso y presiona ENTER.
8. El sistema muestra ventana de confirmación con datos del docente, curso y tipo de registro (ENTRADA/SALIDA). Cuenta regresiva de 10 segundos.
9. Si el docente presiona ENTER: se ejecuta el INSERT o UPDATE en la BD.
10. Se muestra mensaje de éxito durante 2 segundos y se cierra automáticamente.

2. Modelado

2.2 Diagrama de Clases (Descripción textual)

A continuación se describen las clases principales del sistema, detallando sus atributos y métodos:

- **Clase Docente**
  - **Atributos:** `Id` (int), `Codigo` (string), `Nombre` (string), `Apellido` (string), `DNI` (string), `Clase` (string), `Categoria` (string), `Departamento` (string), `Correo` (string).
  - **Métodos:** `ObtenerPorCodigo()` → Docente, `Insertar()` → void, `Actualizar()` → void, `Eliminar()` → void.
- **Clase Curso**
  - **Atributos:** `Id` (int), `Nombre` (string), `Codigo` (string), `Creditos` (int), `Tipo` (string), `Escuela` (string), `MallaCurricular` (string).
  - **Métodos:** `Insertar()` → void, `Actualizar()` → void, `Eliminar()` → void, `BuscarPorNombre()` → List.
- **Clase HorarioClases**
  - **Atributos:** `Id` (int), `DocenteId` (int), `CursoId` (int), `DiaSemana` (int), `HoraInicio` (TimeSpan), `HoraFin` (TimeSpan), `PeriodoId` (int).
  - **Métodos:** `ObtenerPorDia()` → List, `VerificarConflicto()` → bool, `Insertar()` → void, `Actualizar()` → void.
- **Clase RegistroAsistencia**
  - **Atributos:** `Id` (int), `DocenteId` (int), `CursoId` (int), `FechaHora` (DateTime), `HoraSalida` (DateTime?), `PeriodoId` (int).
  - **Métodos:** `RegistrarEntrada()` → void, `RegistrarSalida()` → void, `ObtenerPendientes()` → List, `CalcularHoras()` → decimal.
- **Clase Periodo**
  - **Atributos:** `Id` (int), `Nombre` (string), `EsActivo` (bool).
  - **Métodos:** `ObtenerActivo()` → Periodo, `Activar()` → void.

3. Construcción — Arquitectura de 4 Capas

El software se implementó bajo una arquitectura lógica de 4 capas:

Capa	Responsabilidad	Componentes en el Proyecto
1. Capa de Entidades (Entities)	Define la representación en memoria de las entidades del negocio. Contiene las propiedades que mapean directamente a las columnas de la BD.	Variables miembro en cada Form: `docenteIdActual`, `docenteNombreActual`, `docenteCodigoActual`, `DataTable cursosHoyDT`, etc.
2. Capa de Acceso a Datos (DAL)	Responsable de la comunicación directa con SQL Server. Ejecuta sentencias SQL y devuelve datos crudos.	Conexiones `SqlConnection`, ejecución de `SqlCommand`, lectura con `SqlDataReader` y llenado de `DataTable` con `SqlDataAdapter`.
3. Capa de Lógica de Negocio (BLL)	Implementa las reglas de negocio, validaciones y transformaciones de datos.	Métodos como `DeterminarTipoRegistro()` (decide si es Entrada o Salida), `ObtenerDiaSemanaSQL()` (convierte DayOfWeek a formato BD), validación de conflictos horarios, control secuencial de asistencia.
4. Capa de Presentación (UI)	Interfaz gráfica de usuario. Captura eventos y muestra información al usuario.	`Form1.cs` (pantalla principal del docente), `FormRegistroCurso.cs` (confirmación), `FormAdmin.cs` (panel administrativo con tabs), `FormLogin.cs` (autenticación), `FormEditarHorario.cs`, `FormEditarDocente.cs`, `FormEditarCurso.cs`.

Tecnologías utilizadas:

- **Lenguaje:** C# (.NET Framework 4.7.2)
- **IDE:** Visual Studio
- **Base de Datos:** SQL Server Express (LocalDB)
- **Librería UI:** Guna.UI2 WinForms
- **Exportación:** iTextSharp (PDF)
- **Conexión BD:** ADO.NET (System.Data.SqlClient)

4. Pruebas — Batería de Datos de Prueba

Caso de Prueba CP-01: Registro de Entrada Exitoso

Campo	Valor
Precondición	Docente DOC001 (ALAN TURING) tiene curso "PROGRAMACIÓN BÁSICA" registrado en HorarioClases para el día actual. No tiene registros de asistencia previos hoy.
Entrada	Código: `DOC001`
Acción	Digitar código → Sistema detecta horario → Muestra "[INGRESO] PROGRAMACIÓN BÁSICA" → Presionar ENTER → Confirmar con ENTER
Salida Esperada	INSERT en RegistroAsistencia con DocenteId=1, CursoId=4, FechaHora=hora actual, HoraSalida=NULL. Mensaje: "¡ASISTENCIA REGISTRADA!"
Resultado	[PASÓ]

Caso de Prueba CP-02: Registro de Salida

Campo	Valor
Precondición	Docente DOC001 ya tiene una entrada registrada hoy para "PROGRAMACIÓN BÁSICA" con HoraSalida = NULL.
Entrada	Código: `DOC001`
Acción	Digitar código → Sistema detecta entrada abierta → Muestra "[SALIDA PENDIENTE]: PROGRAMACIÓN BÁSICA" → Presionar ENTER → Confirmar
Salida Esperada	UPDATE RegistroAsistencia SET HoraSalida = hora actual WHERE Id = registro abierto. Mensaje: "¡SALIDA REGISTRADA!"
Resultado	[PASÓ]

Caso de Prueba CP-03: Dos cursos en el mismo día — Validación secuencial

Campo	Valor
Precondición	Docente DOC004 (ALBERT EINSTEIN) tiene CÁLCULO I de 08:00-10:00 y FÍSICA I de 14:00-16:00 hoy. Registró entrada a CÁLCULO I a las 08:05 pero NO registró salida.
Entrada	Código: `DOC004` a las 14:00
Acción	Digitar código → Sistema detecta entrada abierta de CÁLCULO I sin salida
Salida Esperada	El sistema muestra la salida pendiente de CÁLCULO I como prioridad ("[SALIDA PENDIENTE]: CÁLCULO I"). El docente debe cerrar esa sesión antes de poder registrar ingreso a FÍSICA I.

Resultado	[PASÓ]

Caso de Prueba CP-04: Timeout por inactividad

Campo	Valor
Precondición	Se abrió la ventana de registro de curso.
Entrada	No se presiona ninguna tecla durante 20 segundos.
Acción	Esperar 20 segundos sin interacción.
Salida Esperada	La ventana se cierra automáticamente sin registrar nada en la base de datos. El sistema regresa a la pantalla principal.
Resultado	[PASÓ]

Caso de Prueba CP-05: Código de docente inexistente

Campo	Valor
Precondición	No existe docente con código "XXX999" en la BD.
Entrada	Código: `XXX999`
Acción	Digitar código → Presionar ENTER
Salida Esperada	Mensaje de error: "Código no encontrado". No se registra nada.
Resultado	[PASÓ]

Caso de Prueba CP-06: Conflicto de horario al crear horario nuevo

Campo	Valor
Precondición	Docente DOC001 ya tiene PROGRAMACIÓN BÁSICA de 08:00-10:00 el Lunes.
Entrada	Crear nuevo horario: DOC001, BASE DE DATOS, Lunes, 09:00-11:00.
Acción	En FormEditarHorario → Seleccionar datos → Click "Guardar".
Salida Esperada	Mensaje: "El docente ya tiene un horario que se superpone en ese día y hora." No se inserta el registro.
Resultado	[PASÓ]

Caso de Prueba CP-07: Docente sin horario configurado para hoy (modo fallback)

Campo	Valor
Precondición	Docente DOC010 (CARL SAGAN) no tiene ningún horario registrado en HorarioClases para el día actual.
Entrada	Código: `DOC010`

Acción	Digitar código → Sistema no encuentra horarios.
Salida Esperada	Se abre FormRegistroCurso en modo manual (fallback), mostrando un ComboBox con buscador predictivo para que el docente seleccione manualmente su curso.
Resultado	[PASÓ]